

CHAPTER 27

Solar sails

27.1. Introduction

Solar sails are a potential method for traveling within the solar system using the Sun as the propulsion source. The advantage is that no fuel is carried, thus permitting large velocity changes. There are many applications of solar sails. For example, a solar-sail could be sent out of the solar system by first diving at the Sun, passing around the Sun quickly and then heading on a path at very high velocity into deep space. Another application for solar sails is known as a pole sitter where the sail force cancels a planet's and the Sun's gravitational attraction and allows it to be stationary above a planet's pole [1]. This would allow continuous observation of a planet from the polar point of view.

An example of a solar sail is the NASA Near-Earth Asteroid Scout, or NEA Scout, [2,3] is a mission to map an asteroid and demonstrate several technological firsts, including being the first CubeSat to reach an asteroid. NEA Scout is designed to perform reconnaissance of an asteroid using a CubeSat and solar-sail propulsion, which offers navigational agility during the cruise for approaching the target. Using the Sun to propel the spacecraft, NEA Scout (Fig. 27.1) was designed to fly past the 92-m diameter asteroid. NEA Scout could track the asteroid's orbit and measure its shape, rotational properties, spectral class, local dust and debris field, regional morphology, and regolith properties.

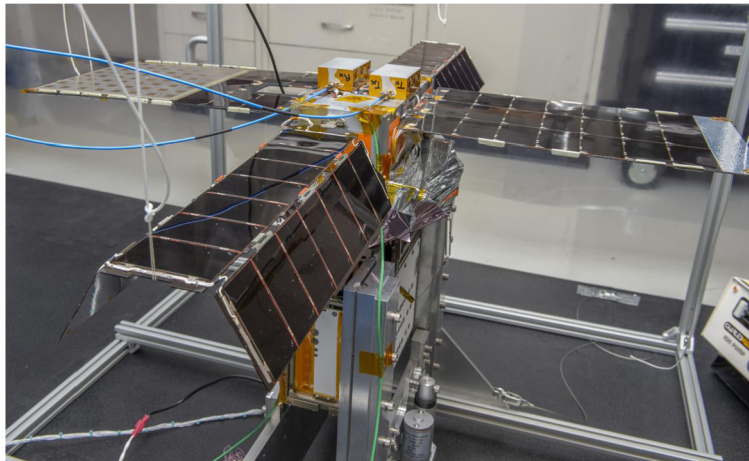


Figure 27.1 The NEA Scout solar-sail spacecraft is shown during integration and test. Image courtesy of NASA.

This chapter describes the design of a solar-sail attitude control system. Fig. 27.2 shows some types of sails. From left, clockwise, control is accomplished by

- Moving the center-of-mass;
- Rotating the vanes to produce control torques;
- Rotating the sail segments to produce control torques;
- Rotating the boom to move the center-of-mass.

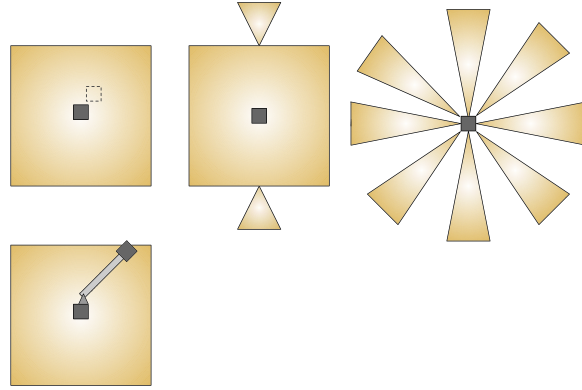


Figure 27.2 Four solar-sail types.

27.2. Gyrostat with a moving mass

Reaction wheels can be used to control a solar sail. A spacecraft with reaction wheels is modeled as a gyrostat, which is a rigid spacecraft, except for symmetric rotors that spin about axes fixed to the body frame. Many spacecraft are gyrostats since reaction wheels can control the spacecraft's attitude without the need to expend propellant. A gyrostat is used to represent spacecraft with reaction wheels. The reaction wheels would quickly saturate if a momentum-control method is not included. In this control system, a moving mass is used to manage momentum.

The momentum for a gyrostat is

$$H = A \left(I\omega + \sum_k u_k J_k (\Omega_k + u_k^T \omega) \right) \quad (27.1)$$

where Ω_k is the angular rate of the k th wheel about its spin-axis, ω are the body rates, J_k is the spin-axis moment of inertia, u_k is the unit vector of the wheel spin-axis measured in the body frame, and I is the inertia of the rigid body excluding the wheels.

The equations of motion are then

$$T = I\dot{\omega} + \omega^\times \left(I\omega + \sum_k u_k J_k (\Omega_k + u_k^T \omega) \right) + \sum_k u_k J_k (\dot{\Omega}_k + u_k^T \dot{\omega}) \quad (27.2)$$

$$T_k = J_k (\dot{\Omega}_k + u_k^T \dot{\omega}) \quad (27.3)$$

The transverse axes' inertias are lumped into the core body inertia. T is the body-fixed torque on the spacecraft and T_k is the scalar torque on each wheel. These equations give $3 + n$ equations of motion. To integrate these equations, you need first to solve for the angular acceleration of the core, and then substitute the angular-acceleration vector into the reaction-wheel equations,

$$\dot{\omega} = I^{-1} \left(T - \omega^\times \left(I\omega + \sum_k u_k J_k (\Omega_k + u_k^T \omega) \right) - \sum_k u_k T_k \right) \quad (27.4)$$

$$\dot{\Omega}_k = \frac{T_k}{J_k} - u_k^T \dot{\omega} \quad (27.5)$$

Momentum is controlled using a moving mass. The moving-mass model is depicted in Fig. 27.3. The first image shows that the model has two sliders, each with one degree-of-freedom (e.g., body 2 is fixed to stepper 2), as well as the track geometry for the NEAR Scout spacecraft. The second figure shows the dynamic quantities.

Table 27.1 gives the definitions of the variables. The core body, 1, has the reaction wheels.

Table 27.1 Moving-mass variables.

Variable	Definition	Constant?
B_1	Transformation matrix from slider 1 to the core body	Yes
B_2	Transformation matrix from slider 2 to slider 1	Yes
B	Transformation matrix from body 2 to body 1	No
I_1	Body 1 inertia matrix	Yes
I_2	Body 2 inertia matrix	Yes
m_1	Body 1 mass	Yes
m_2	Body 2 mass	Yes
A	Transformation matrix from body 1 to the inertial frame (ECI)	No
ω_1	Angular rate vector of body 1 with respect to the ECI frame measured in the body-1 frame	No
ω_2	Total angular rate vector of body 2 with respect to the ECI frame measured in the body-2 frame	No
Ω_2	Total angular rate vector of body 2 with respect to body 2 measured in the body 2 frame	No

continued on next page

Table 27.1 (continued)

Variable	Definition	Constant?
D_1	Vector from the total center-of-mass to the center-of-mass of body 1 written in the ECI frame	No
D_2	Vector from the total center-of-mass to the center-of-mass of body 2 written in the ECI frame	No
d_1	Vector from the total center-of-mass to the center-of-mass of body 1 written in the body 1 frame	No
d_2	Vector from the total center-of-mass to the center of mass of body 2 written in the body 1 frame	No
r_1	Vector from the reference point of body 1 written in the body-1 frame	No
r_2	Vector from the reference point of body 1 to body 2 written in the body-1 frame	No
u_1	Slider 1 unit vector	Yes
u_2	Slider 2 unit vector	Yes
ρ_1	Distance along slider 1 unit vector	No
ρ_2	Distance along slider 2 unit vector	No
λ_1	Distance from reference to slider 1 origin	Yes
λ_2	Distance from slider 2 tip to center-of-mass of body 2	Yes
c	Center-of-mass in the body 1 frame	No

Fig. 27.3 shows the two bodies that are connected via two sliders. Each slider has a single degree-of-freedom that we define as always along the z -axis of the slider. The transformation matrix from slider 1 to the core is B_1 and from slider 2 to slider 1 is B_2 . r goes from the origin to the mass. X is from the inertial origin to the spacecraft center-of-mass (which may move). d goes from the spacecraft center-of-mass to the moveable mass.

The total angular momentum of the vehicle is

$$H = AI_1\omega_1 + ABI_2\omega_2 + m_1D_1^\times \dot{D}_1 + m_2D_2^\times \dot{D}_2 \quad (27.6)$$

where I_1 is the inertia with respect to the center-of-mass of the core, I_2 is the inertia matrix with respect to the center-of-mass of body 2, D_k are in the inertial frame, and A is the transformation matrix from the body to the inertial frame. $B = B_1B_2$.

The notation $a^\times$ is the matrix equivalent of the crossproduct

$$a^\times = \begin{bmatrix} 0 & -a_x & a_y \\ a_x & 0 & -a_z \\ -a_y & a_z & 0 \end{bmatrix} \quad (27.7)$$

$$D_1 = Ad_1 \quad (27.8)$$

$$D_2 = ABd_2 \quad (27.9)$$

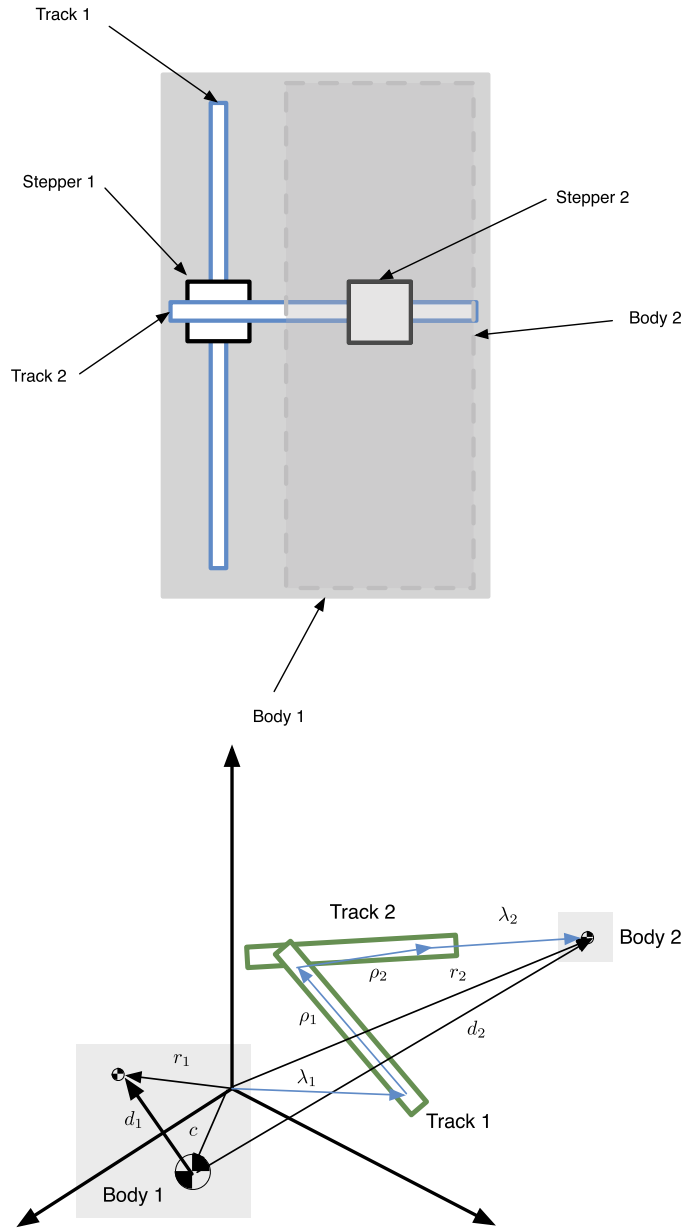


Figure 27.3 Moving-mass geometry.

where B is a constant and

$$d_k = r_k - c \quad (27.10)$$

where c is the location of the center-of-mass and r_1 is a constant,

$$r_2 = \lambda_1 + B_1 u_1 \rho_1 + B_1 B_2 (u_2 \rho_2 + \lambda_2) \quad (27.11)$$

and ρ_k is a scalar. λ_1 and λ_2 are constants.

The center-of-mass c is

$$c = \frac{1}{M} (m_1 r_1 + m_2 r_2) \quad (27.12)$$

where M is the total mass, or

$$c = \mu_1 r_1 + \mu_2 r_2 \quad (27.13)$$

where $\mu_k = \frac{m_k}{M}$.

We can then write

$$d_1 = r_1 - c = r_1(1 - \mu_1) - \mu_2 r_2 \quad (27.14)$$

$$d_1 = \mu_2(r_1 - r_2) \quad (27.15)$$

$$d_2 = \mu_1(r_2 - r_1) \quad (27.16)$$

The derivatives are

$$\dot{d}_1 = -\mu_2 \dot{r}_2 \quad (27.17)$$

$$\dot{d}_2 = \mu_1 \dot{r}_2 \quad (27.18)$$

$$\dot{r}_2 = B_1 (u_1 \dot{\rho}_1 + B_2 u_2 \dot{\rho}_2) \quad (27.19)$$

The accelerations are

$$\ddot{d}_1 = -\mu_2 \ddot{r}_2 \quad (27.20)$$

$$\ddot{d}_2 = \mu_1 \ddot{r}_2 \quad (27.21)$$

$$\ddot{r}_2 = B_1 (u_1 \ddot{\rho}_1 + B_2 u_2 \ddot{\rho}_2) \quad (27.22)$$

$$\dot{D}_1 = A(\omega^\times d_1 + \dot{d}_1) \quad (27.23)$$

$$\dot{D}_2 = A(\omega^\times d_2 + \dot{d}_2) \quad (27.24)$$

$$\ddot{D}_1 = A(\omega^\times \omega^\times d_1 + 2\omega^\times \dot{d}_1 + \ddot{d}_1 - d_1^\times \dot{\omega}) \quad (27.25)$$

$$\ddot{D}_2 = A(\omega^\times \omega^\times d_2 + 2\omega^\times \dot{d}_2 + \ddot{d}_2 - d_2^\times \dot{\omega}) \quad (27.26)$$

We need to find the three core dynamical equations for the entire spacecraft and the two scalar equations for the masses. Taking the derivative of Eq. (27.6) we get

$$T = I\dot{\omega} + \omega^\times I\omega + m_1 d_1^\times \left(\omega^\times \omega^\times d_1 + 2\omega^\times \dot{d}_1 + \ddot{d}_1 - d_1^\times \dot{\omega} \right) + m_2 d_2^\times \left(\omega^\times \omega^\times d_2 + 2\omega^\times \dot{d}_2 + \ddot{d}_2 - d_2^\times \dot{\omega} \right) \quad (27.27)$$

where $I = I_1 + BI_2B^T$, $\omega = \omega_1$ and using the identity

$$(Ad_k)^\times = Ad_k^\times A^T \quad (27.28)$$

The final step is to get the scalar equations for the movement of the masses. We write

$$Au_1 f_1 = m_1 \ddot{D}_1 + m_2 \ddot{D}_2 \quad (27.29)$$

$$Au_2 f_2 = m_k \ddot{D}_2 \quad (27.30)$$

For the outer link

$$u_k f_2 = m_2 \left(\omega^\times \omega^\times d_2 + 2\omega^\times \dot{d}_2 + \ddot{d}_2 - d_2^\times \dot{\omega} \right) \quad (27.31)$$

Multiplying both sides by u_2^T

$$f_2 = m_2 u_2^T \left(\omega^\times \omega^\times d_2 + 2\omega^\times \dot{d}_2 + \ddot{d}_2 - d_2^\times \dot{\omega} \right) \quad (27.32)$$

which is a scalar and

$$T = I\dot{\omega} + \omega^\times I\omega + m_1 d_1^\times u_1 f_1 + m_2 d_2^\times u_2 f_2 \quad (27.33)$$

27.3. Thin-membrane model

Since the sail is a thin membrane, we can make the additional assumption that it is at a constant temperature, therefore we can perform a flux balance and combine the optical and thermal forces into one model. The flux balance is illustrated in Fig. 27.4. Each side of the membrane, front and back, has its emissivity (ϵ).

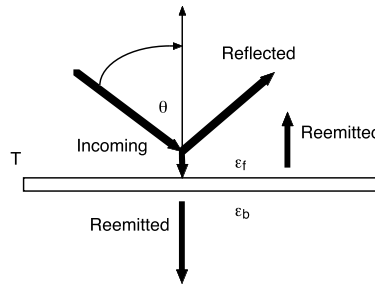


Figure 27.4 Flux balance on the solar-sail surface.

In steady state, the incoming absorbed solar flux must equal the outgoing reemitted flux, which follows Boltzmann's law for thermal radiation. Table 27.2 gives the definitions of each of the variables used:

Table 27.2 Solar Sail Variables.

Variable	Definition	Constant
ρ_a	Absorption coefficient	Yes
P	Incoming solar flux, $P = \Phi \cos(\theta)$	No
σ	Stefan–Boltzmann constant ($5.670 \times 10^{-8} \text{ J K}^{-4} \text{ s}^{-1}$)	Yes
ϵ_f	Emissivity of front of sail	Yes
ϵ_b	Emissivity of back of sail	Yes
T_s	Temperature of the front and back sails (assumed to be equal)	No

We can write:

$$\rho_a P = \sigma (\epsilon_f T_s^4 + \epsilon_b T_s^4) \quad (27.34)$$

We can solve for the equilibrium temperature as:

$$T_s = \left(\frac{\rho_a P}{\sigma (\epsilon_f + \epsilon_b)} \right)^{1/4} \quad (27.35)$$

The reemitted fluxes per unit area for the front and back of the membrane are independent of temperature:

$$\Phi_f = \frac{\epsilon_f \rho_a P}{\epsilon_f + \epsilon_b} \quad (27.36)$$

$$\Phi_b = \frac{\epsilon_b \rho_a P}{\epsilon_f + \epsilon_b} \quad (27.37)$$

The remaining outgoing fluxes accounting for the specular and diffuse reflected portions of the solar flux are:

$$\Phi_s = \rho_s P \quad (27.38)$$

$$\Phi_d = \rho_d P \quad (27.39)$$

From the conservation of energy, we know that these four outgoing fluxes must equal the incoming flux. The total force exerted on the sail is:

$$F = \cos(\theta) (f_s + f_d + f_f + f_b) \quad (27.40)$$

where the normalized force contributions due to each flux are:

$$f_s = -2 \cos(\theta) \Phi_s \hat{n} \quad (27.41)$$

$$\begin{aligned}
f_d &= -\Phi_d \left(\frac{2}{3} \hat{n} + \hat{s} \right) \\
f_f &= -\frac{2}{3} \Phi_f \hat{n} + \Phi_f \hat{s} \\
f_b &= \frac{2}{3} \Phi_b \hat{n} + \Phi_b \hat{s}
\end{aligned} \tag{27.42}$$

Combining terms, we get the final form of the thermal/optical force model:

$$F = -pA \cos(\theta) \left(2 \left[\rho_s \cos(\theta) + \frac{1}{3} \left(\rho_d + \rho_a \frac{\epsilon_f - \epsilon_b}{\epsilon_f + \epsilon_b} \right) \right] \hat{n} + [\rho_d + \rho_a] \hat{s} \right) \tag{27.43}$$

To express the force in terms of components normal and tangential to the sail, we note that the Sun unit vector can be expressed as $\hat{s} = \cos(\theta) \hat{n} + \sin(\theta) \hat{t}$. Therefore,

$$f_n = pA \cos(\theta) \left((1 + \rho_s) \cos(\theta) + \frac{2}{3} \left(\rho_d + \rho_a \frac{\epsilon_f - \epsilon_b}{\epsilon_f + \epsilon_b} \right) \right) \tag{27.44}$$

$$f_t = pA \cos(\theta) \sin(\theta) (\rho_d + \rho_a) \tag{27.45}$$

27.4. Momentum control

Momentum control is done either with thrusters or with the moving-mass system.

The momentum-control equation is a proportional-integral (PI) controller.

$$\dot{h} = -K \left(h + \frac{1}{\tau} \int h \right) \tag{27.46}$$

where the momentum h and torque T are in the inertial frames. The gain and integral time constant are set to maintain wheel margins and provide damping. When thrusters are used the computation of torque is straightforward.

When the moving mass is used we have the equation for the body-frame torque due to the moving mass

$$T = (c_p - c(\rho))^\times F \tag{27.47}$$

where F is the sail force in the body frame. We only have control over two elements of ρ .

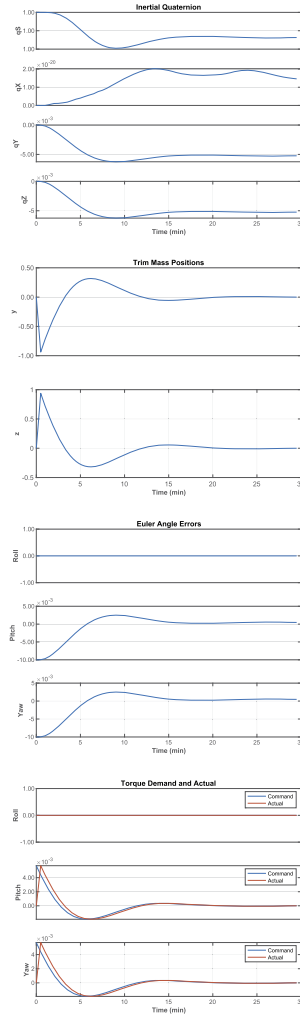
We solve this by using the MATLAB[®] function `fminsearch` to find the two positions that best fit the torque using the center-of-mass calculation in Eq. (27.12). `fminsearch` does not apply constraints. If the demand on ρ is at a limit it is set to a limit.

A standalone momentum-control demo shows the results given in Example 27.1.

```

1 % Constants
2 clear d
3 c = 3e8;
4 pSun = 1367;
5 wSail = 100;
6 % User Parameters
7 pitchStep = 0.01; % y-axis, radians
8 yawStep = 0.01; % z-axis, radians
9 g = load('PlateWithMasses');
10
11 %% Design controller
12 xN = zeros(6,1);
13 iner = diag([1 1 1]);
14 zeta = 2*ones(1,3);
15 wn = 0.001*ones(1,3);
16 tauInt = 5000*ones(1,3);
17 omegaR = 5*wn;
18 dT = 30;
19 [aC,bC,cC,dC] = PIDMIMO( iner, zeta, wn, tauInt, omegaR, dT);
20 uControl = [0 1 0; 0 0 1]';
21 dOffset = zeros(3,3);
22 mControl = [g.body(1).mass, mass, g.body(2).mass, mass, g.body(3).mass, mass];
23
24 % States for each body, including attitude, are stacked
25 xCore = [zeros(6,1); QZero; zeros(3,1)];
26 xMass = [zeros(6,1); QZero; zeros(3,1)];
27 x = [xCore; xMass; xMass];
28 % Indices
29 iR1 = 2+13; % y mass position
30 iR2 = 3+26; % z mass position
31 iV1 = 5+13; % y mass velocity
32 iV2 = 6+26; % z mass velocity
33 % New attitude command
34 angCommand = [0; pitchStep; yawStep];
35 % Assume a constant sail force in the ECI frame
36 f = struct;
37 f.total = [-2*pSun/c*wSail^2; 0; 0];
38 tq = struct;
39 tq.total = [0; 0; 0];
40 % Center of pressure
41 Cp = [0; 0; 0];
42 % Additional fields for RHS
43 d.g = g;
44 d.nBody = 3;
45 maxRate = 1; % m/s
46 % Number of simulation points
47 nSim = 60;
48 xP = zeros(size(x,1)+9,nSim);
49
50 % Simulation loop
51 for k = 1:nSim
52 % Transverse angle error (small angles)
53 u = QTForm( x(7:10), [1;0;0] );
54 angleError = [0;-u(3)-angCommand(2); u(2)-angCommand(3)];
55
56 % Control
57 sN = c*xN + cAngleError;
58 xN = aC*sN + bC*angleError;
59 Tcommand = -g.mass.inertia*sN;
60 cM = TorqueToCM( Tcommand, f.total, Cp );
61 rhoCommand = CMToMassPositions( cM, mControl, dOffset, uControl );
62 rhoActual = [x(iR1); x(iR2)];
63 deltaRho = rhoCommand - rhoActual;
64 rhoDot = sign(deltaRho)*min(abs(deltaRho)/dT, maxRate*[1;1]);
65
66 % Update rates
67 xRates = x;
68 xRates(iV1) = rhoDot(1);
69 xRates(iV2) = rhoDot(2);
70 xNew = FMovingBody( 'init', x, xRates, tq, struct('g', g) );
71
72 % Disturbances - simple torque model
73 g.body(2).rHinge = [0; rhoActual(1); 0];
74 g.body(3).rHinge = [0; 0; rhoActual(2)];
75 cMActual = (mControl(2)*g.body(2).rHinge + mControl(3)*g.body(3).rHinge)/g.mass.mass;
76
77 tq.total = Cross( Cp - cMActual, f.total );
78 d.force = f;
79 d.torque = tq;
80
81 % Store
82 xP(:,k) = [x; angleError; Tcommand; tq.total];
83
84 % Integrate
85 x = RK4( @FCoreAndMoving, xNew, dT, 0, '', d );
86 end
87
88 % Plot
89 i = (0:nSim-1)*dT;
90 rPY = ['Roll', 'Pitch', 'Yaw'];
91 TimeHistory(i, xP(7:10,:), {'qS', 'qX', 'qY', 'qZ'}, 'Quaternion');
92 TimeHistory(i, xP([iR1 iR2]), {'y', 'z'}, 'TrimMass');
93 TimeHistory(i, xP(40:42,:), rPY, 'AngleErrors');
94 i = ['Command', 'Actual'];
95 TimeHistory( i, xP(43:48,:), rPY, ...
96 'Torque_Demand_and_Actual', [1 4] [2 5] [3 6]), (1 1 1));
97
98 % Figure

```



Example 27.1: Control with moving masses.

27.5. Attitude control

This attitude control system example uses a proportional-integral-differential (PID) controller with reaction wheels for control and vanes for momentum unloading. Vanes are placed at the corners of the sail and have a single axis of rotational freedom. The configuration is shown in Fig. 27.5.

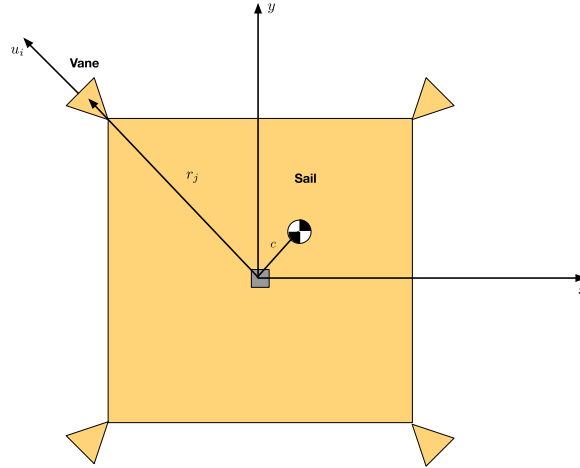


Figure 27.5 Sail with vanes. c is the vector from the geometric center to the center-of-mass. u_i is the rotation axis for the vanes.

The core control system will use a PID controller.

$$\frac{u_c(s)}{\theta(s)} = K_p \left(1 + \tau_d s + \frac{1}{\tau_i s} \right) \quad (27.48)$$

where τ_d is the rate time constant, which is how long the system will take to damp and τ_i is how fast the system will cancel a steady disturbance. The controller will drive the reaction wheels. The momentum controller will use a proportional-integral (PI) controller.

$$\frac{T_c(s)}{H(s)} = K_p \left(1 + \frac{1}{\tau_i s} \right) \quad (27.49)$$

The controller will command vane angles.

The torque on the sail and the vanes, assuming perfect specular reflection is

$$t = -2p \sum_{i=1}^N a_i (s^T n_i)^2 (r_i - c) \times n_i \quad (27.50)$$

where p is the solar pressure, which is the solar flux divided by the speed-of-light, a_i is the area of each surface, and n_i is the normal for each surface. The normal vector for

the main sail is along $+z$. The normal vectors for a vane, as a function of vane angle is

$$n_i = \begin{bmatrix} \cos \alpha_i & \sin \alpha_i & 0 \\ -\sin \alpha_i & \cos \alpha_i & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta_i & 0 & \sin \beta_i \\ 0 & 1 & 0 \\ -\sin \beta_i & 0 & \cos \beta_i \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (27.51)$$

This simplifies to

$$n_i = \begin{bmatrix} \cos \alpha_i \sin \beta_i \\ -\sin \alpha_i \sin \beta_i \\ \cos \beta_i \end{bmatrix} \quad (27.52)$$

Linearized about small β_i ,

$$n_i = \begin{bmatrix} \beta_i \cos \alpha_i \\ \beta_i \sin \alpha_i \\ 1 \end{bmatrix} \quad (27.53)$$

where β_i are the control angles.

The performance of vanes can be understood by looking at the torque-balancing problem. To unload momentum, the vanes need to be able to balance the net torque on the sail. Example 27.2 shows an example of torque balancing. The area of the vanes must

```

1 %% Momentum control with vanes
2 r = 50; % m
3 d.r = r*[-1 1 -1 1 0;...
4         1 1 -1 -1 0;...
5         0 0 0 0 0];
6 d.alpha = [pi/4 3*pi/4 5*pi/4 7*pi/4 0];
7 d.area = [4 4 4 4 100*100];
8 d.c = [0.02;0.2;0.1];
9 d.uSun = [0;0;1];
10
11 beta = [0;0;0;0];
12 t0 = TorqueVane(beta,d);
13
14 opts = optimset('fminsearch');
15
16 beta = fminsearch(@FVanes,beta,[],d);
17 tF = TorqueVane(beta,d);
18
19 disp('Torque')
20 fprintf('Initial [%8.1e %8.1e %8.1e] micro-Nm\n',
21         t0*1e6);
22 fprintf('Optimal [%8.1e %8.1e %8.1e] micro-Nm\n',
23         tF*1e6);
24 fprintf('Angles [%5.2f %5.2f %5.2f] deg\n',
25         beta)*180/pi;
26
27 %% Function called by fminsearch
28 function y = FVanes(beta,d)
29
30     t = TorqueVane(beta,d);
31     y = Mag(t);
32 end

```

```

1 Torque
2 Initial [ 9.1e+03 -9.1e+02 0.0e+00] micro-Nm
3 Optimal [-2.6e-02 -5.9e-03 -7.0e-03] micro-Nm
4 Angles [-0.93 -0.44 0.03 1.39] deg

```

Example 27.2: Vanes for torque balancing.

be sufficient to balance the torque due to the offset of the mainsail center-of-pressure from the spacecraft center-of-mass.

27.6. Architecture

The attitude control system is shown in Fig. 27.6. The blue boxes are actuator hardware. The green boxes are sensor hardware. The black boxes are included in the default attitude control system. The gray boxes, needed for a flight system, are not included. IMU stands for inertial measurement unit. It includes both gyros and linear accelerometers. For this application, only the gyros are used.

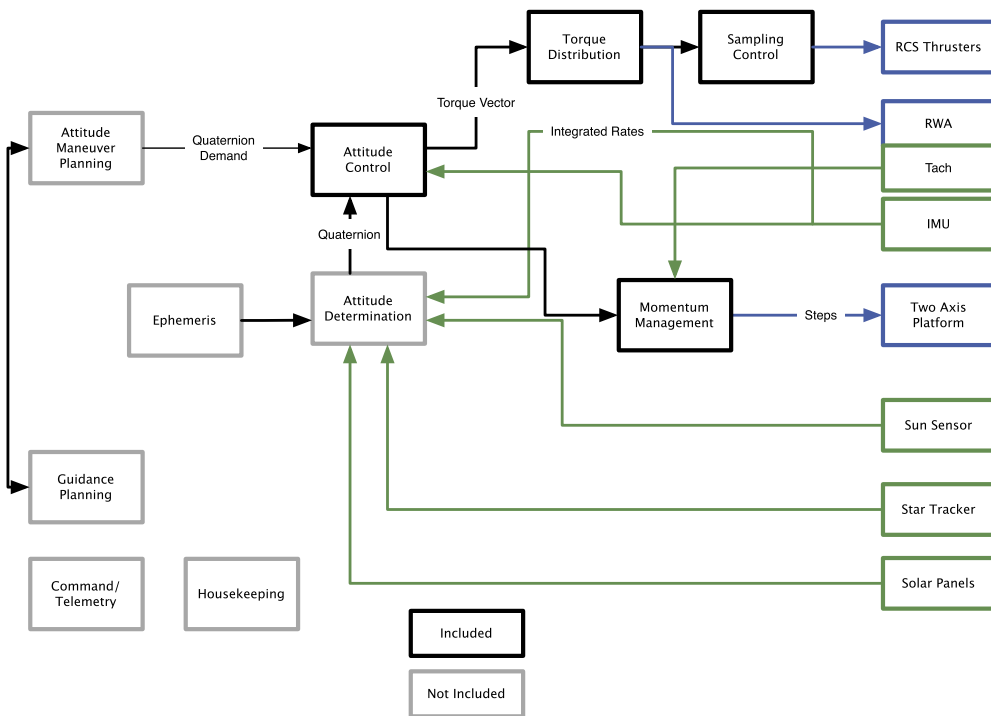


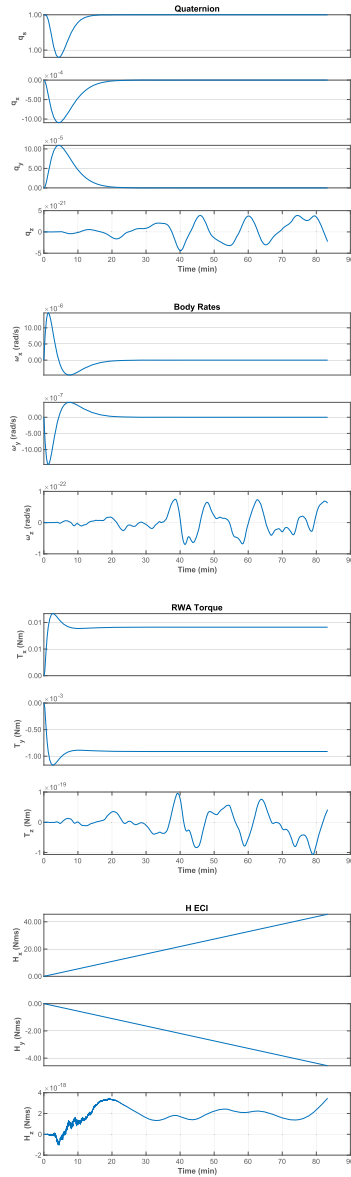
Figure 27.6 GN&C system. The black boxes are included ACS software. IMU stands for inertial measurement unit.

Example 27.3 shows the sail simulation. The true quaternion is used as the attitude reference. The attitude determination system is not simulated. Fig. 27.7 shows the simulation with the momentum-management system enabled. The algorithm for computing the β angles uses the last value as a starting point. This results in smooth operation. Since it quickly nulls the net torque it is constant. This assumes that it has a good model of the sail. In practice, there will be errors and some momentum will build up and have

```

1 %% Constants
2 controlOn = true;
3 nmOn      = true; % Momentum management
4 rToD     = 180/pi;
5
6 %% User initialization
7 n        = 5000; % Number of simulation steps
8 dT       = 1.0; % Time step (s)
9
10 % Momentum Controller
11 omegaC   = 0.1; % Control bandwidth
12 zetaC    = 1; % Control damping ratio
13
14 % Initial states
15 q        = [1;0;0;0];
16 omega    = [0;0;0];
17 omegaRWA = [0;0;0];
18 uSun     = [0;0;1];
19 r        = 50; % m
20 dV.r     = r*[ -1 1 -1 1 0;...
21              1 1 -1 -1 0;...
22              0 0 0 0 0];
23 dV.alpha = [pi/4 3*pi/4 5*pi/4 7*pi/4 0];
24 dV.area  = [20 20 20 20 100*100];
25 dV.c     = [0.02;0.2;0.1];
26 dV.uSun  = uSun;
27
28 %% State vector
29 x        = [q;omega;omegaRWA];
30 dRHS     = RHSGyrostat;
31 dRHS.inr = diag([25000 25000 50000]);
32
33 %% Simulate
34 xP       = zeros(length(x)+13,n);
35 % Edit the control system data structure
36 dC       = SailACS;
37 dC.inr   = dRHS.inr;
38 dC.dT    = dT;
39 dC.omega = omega; % The LVLH rate
40 dC.nmOn  = nmOn;
41 dC.kF    = dRHS.inr(2,2)*omegaC^2;
42 dC.tauR  = 2*omegaC*zetaC*dRHS.inr(2,2)/dC.kF;
43 dC.vanes = dV;
44 dC.uSun  = uSun;
45
46 % Create a time vector
47 t        = (0:n-1)*dT;
48
49 % Simulation loop
50 for k = 1:n
51     % Update the controller
52     if ( controlOn )
53         [dC,beta,rWA] = SailACS(x,dC);
54     else
55         beta = [0;0;0;0]; rWA = [0;0;0];
56     end
57     dV.uSun = QForm(x(1:4),uSun);
58     tSail = TorqueVane(beta,dV);
59
60     dRHS.torque = tSail; % Nm
61     dRHS.torqueWheel = rWA;
62     [~,hECI] = RHSGyrostat(x,0,dRHS);
63     xP(:,k) = [x;rWA;beta*rToD;hECI;tSail];
64     x = RK4(@RHSGyrostat,x,dT,0,dRHS);
65 end
66
67 %% Plotting
68 yL = {'q_s' 'q_x' 'q_y' 'q_z' ...
69       '\omega_x(rad/s)' '\omega_y(rad/s)' '\omega_z(rad/s)' ...
70       '\Omega_1' '\Omega_2' '\Omega_3' ...
71       'T_x(Nm)' 'T_y(Nm)' 'T_z(Nm)' ...
72       '\beta_1(deg)' '\beta_2(deg)' '\beta_3(deg)' ...
73       '\beta_4(deg)' ...
74       'H_x(Nms)' 'H_y(Nms)' 'H_z(Nms)' ...
75       'T_x(Nm)' 'T_y(Nm)' 'T_z(Nm)'};
76
77 tL = {'Quaternion' 'BodyRates' 'WheelRates' ...
78       'RWA_Torque' 'beta' 'H_ECI' 'Sail'};
79 kL = [1:4 5:7 8:10 11:13 14:17 18:20 21:23];
80 for j = 1:length(tL)
81     k = kL{j};
82     TimeHistory(t,xP(k,:),yL(k),tL{j});
83 end
84 Figure

```



Example 27.3: Attitude control simulation with vanes for momentum control. Momentum control is not enabled.

to be removed. The control system does not feed forward the solar torque. The attitude controller is fast enough to make that unnecessary.

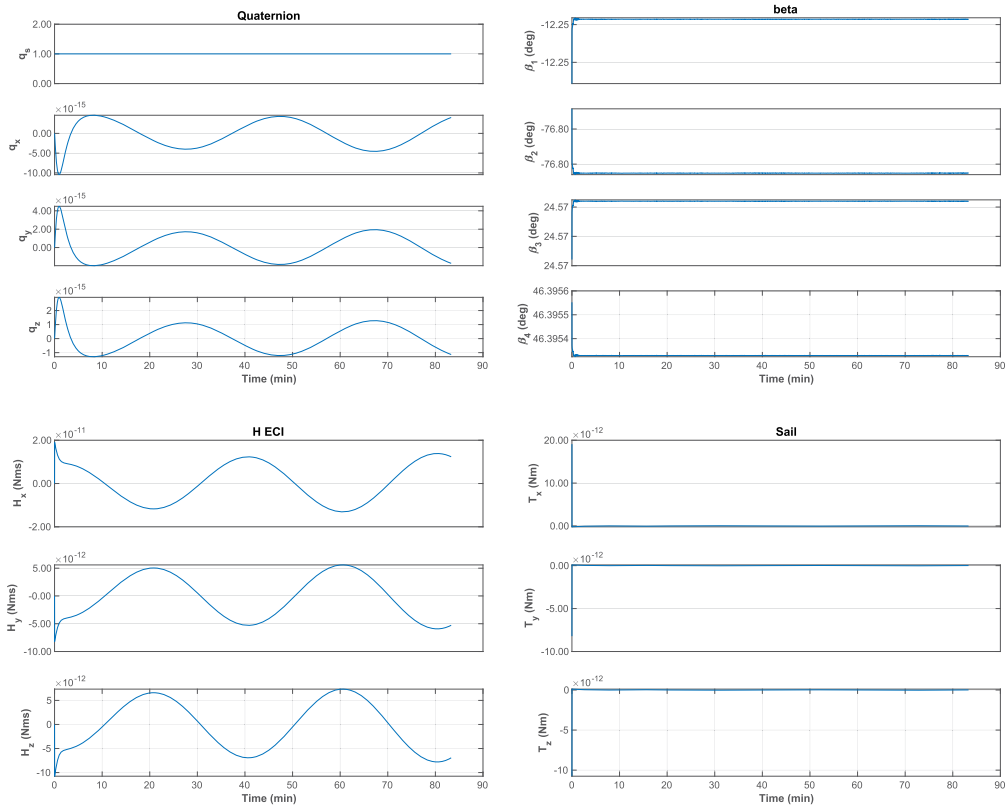


Figure 27.7 Attitude control simulation with vanes enabled for momentum control. The large angles necessitate the use of a numerical method to solve for the angles.

References

- [1] J. Heiligers, M. Vergaaij, M. Ceriotti, End-to-end trajectory design for a solar-sail-only pole-sitter at Venus, Earth, and Mars, in: 5th International Symposium on Solar Sailing, ISSS 2019; Conference date: 30-07-2019 through 02-08-2019, Advances in Space Research 67 (9) (2021) 2995–3011.
- [2] JPL, Near Earth Asteroid Scout (NEAScout), <http://www.jpl.nasa.gov/cubesat/missions/neascout.php>, 2016.
- [3] A. Heaton, J. Castillo-Rogez, L. Johnson, L. Jones, Near Earth asteroid rendezvous mission, in: AIAA Space 2014, August 2014.